

aud-look

Local hybrid search over diarized podcast conversations

Multimodal AI: Audio Search | September 2026

6 episodes	52:29 audio corpus	0.833 holdout Recall@5	565 ms warm p95
----------------------	------------------------------	----------------------------------	---------------------------

A reproducible Python/PostgreSQL system combining literal search, local semantic embeddings, candidate fusion, and local cross-encoder reranking. Results retain the source file, speaker, timestamp, and transcript evidence required for an auditable audio-search experience.

Submission contents

Engineering design and rationale; success criteria and measured achievement; evaluation methodology; production considerations; limitations; reproduction instructions; and AI-usage disclosure.

Effective Retrieval from Audio Transcripts

Submission overview

This project implements local hybrid search over six two-speaker podcast clips. A user can search for exact spoken words, entities, numbers, or semantically related ideas and receive a ranked result containing the episode, source file, diarized speaker, timestamp, matching utterance, and neighboring transcript context.

The system uses hosted Gemini transcription because the challenge permits hosted speech processing. Embeddings, PostgreSQL indexing, semantic search, candidate fusion, and cross-encoder reranking run locally. The complete application is reproducible through Docker Compose and includes a FastAPI service, a Streamlit demonstration interface, cached canonical transcripts, database migrations, automated tests, and a timestamp-grounded evaluation set.

Dataset

The golden corpus contains 52 minutes and 29 seconds of audio across six clips. Every episode contains two intended speakers. Gemini's original diarization labels are preserved per episode as `spk : 0` and `spk : 1`; they identify distinct voices within a file rather than global speaker identities.

Episode	Duration	Utterances	Semantic chunks	Main concepts
Janice Bryant Howroyd	9:19	71	22	discipline, filtering ideas, 12-month outcomes
Jason Fried	8:28	82	22	profitability, VC, autonomy, sustainable growth
Manfredi Lefebvre d'Ovidio	8:29	58	21	money, freedom of choice, time, entrepreneurship
Jordan Noone	8:23	45	20	rockets, Relativity Space, fear, applying skills
Geoffrey Kent	8:34	73	21	instinct, luxury travel, Harrods, entrepreneurship
Shaahin Cheyne	9:16	66	24	Amazon, persistence, apprenticeship, trend spotting
Total	52:29	395	130	

Audio files and metadata are recorded in `dataset/manifest.yaml`. SHA-256 checksums make the corpus stable across transcription, ingestion, and evaluation runs.

Engineering design

```
MP3 files
-> Gemini 3.5 Transcribe
    verbatim text + speaker labels + word timestamps
-> canonical cached word transcript
-> speaker-attributed utterances
    -> PostgreSQL FTS and exact literal candidates
-> overlapping conversational chunks
    -> local BGE embeddings and pgvector candidates
-> union and deduplicate utterance candidates
-> local cross-encoder reranking
-> episode + file + speaker + timestamp + text + context
```

Transcription and normalization

Gemini 3.5 Transcribe runs in verbatim mode with English language guidance, speaker diarization, and word-level timestamps. Each successful response is cached as canonical JSON keyed by the audio checksum and transcription configuration. Repeated ingestion therefore does not consume transcription quota.

Word timestamps are retained because they allow deterministic rebuilding of utterances and chunks without retranscription. The normalizer permits overlapping timestamps caused by cross-talk, sorts annotations by start time, rejects genuine time reversals or out-of-bounds timestamps, and requires exactly two final speaker labels. Utterances end on speaker changes, useful sentence boundaries, or conservative duration and word-count caps.

Storage and indexing

PostgreSQL is the single datastore. The schema contains:

- `audio_files` for stable episode metadata and transcription provenance;
- `utterances` for precise speaker and timestamp evidence;
- `semantic_chunks` for contextual local embeddings; and
- `chunk_utterances` for the ordered relationship between retrieval chunks and result utterances.

A generated English `tsvector` and GIN index support lexical retrieval. Exact matching protects quoted phrases, single terms, names, acronyms, and numbers. `BAAI/bge-base-en-v1.5` creates normalized 768-dimensional embeddings locally. The query prefix recommended by BGE is applied to queries only.

Chunks contain complete consecutive utterances, generally 100-160 words or 30-50 seconds, with one-utterance overlap. Exact cosine scans are used for the small evaluation corpus, avoiding approximate-nearest-neighbor recall loss. An HNSW cosine index is included as the documented scale-out path.

Retrieval and ranking

Lexical and semantic retrieval run concurrently:

1. The lexical branch retrieves up to 20 utterances with PostgreSQL full-text search plus literal matching.
2. The semantic branch retrieves up to 10 conversational chunks by cosine similarity and projects those chunks back to their utterances.
3. Candidates are unioned and deduplicated while retaining lexical and semantic provenance.
4. Up to 60 candidates are reranked locally with `cross-encoder/ms-marco-MiniLM-L-6-v2`.

The public strategies are:

Strategy	Behavior
<code>lexical</code>	Full-text and literal utterance retrieval
<code>semantic</code>	Vector chunk retrieval projected to utterances
<code>rrf</code>	Reciprocal-rank fusion of lexical and semantic candidates
<code>hybrid</code>	Candidate union followed by target-utterance-only reranking
<code>hybrid_alt</code>	Target reranking plus a controlled previous-question/current-answer pair score

`hybrid` is the default. Earlier contextual reranking included previous and next utterances around a target. Testing showed that a host question could incorrectly inherit relevance from a guest answer in the next utterance. Target-only reranking produced better attribution and better holdout retrieval.

hybrid_alt tests a narrower use of context. A response receives an additional question-answer pair score only when its preceding utterance is a detected question from the other speaker and begins no more than seven seconds earlier. The question never receives the answer as future context. This alternative achieved the same relevance metrics as the default but required more reranking work, so it remains experimental.

Quoted phrases and single-token searches receive strict literal priority. Natural-language questions use model relevance rather than absolute phrase priority, preventing a verbatim host question from automatically outranking its answer.

API and demonstration UI

FastAPI exposes health, episode-listing, and search endpoints. Search responses include result rank, episode ID and title, filename, diarized speaker, start and end milliseconds, matching text, neighboring context, retrieval provenance, and literal-match status. Streamlit provides query, strategy, and result-count controls plus readable result cards.

Definition of success

Criterion	Target	Result	Status
Corpus ingestion	Six valid two-speaker episodes	6/6 episodes; 395 utterances	Achieved
Search evidence	Every result has a valid episode, speaker, and timestamp	Enforced by schema, ingestion validation, and integration tests	Achieved
Holdout hybrid Recall@5	At least 0.80	0.833	Achieved
Baseline comparison	Hybrid no worse than strongest lexical/semantic baseline	0.833 vs semantic 0.500	Achieved
Exact/entity HitRate@3	At least 0.90	1.00 on reviewed holdout exact/entity queries	Achieved
Warm p95 hybrid latency	At most 2 seconds on the reference machine	565 ms	Achieved
Deterministic automated checks	All normal tests pass	16 passed	Achieved
Secret handling	No key or remote file identifier in artifacts/logs	Key supplied only through .env; cached artifacts scanned	Achieved

Evaluation methodology

The reviewed golden set contains 36 queries, six per episode. Each episode contributes an exact phrase, semantic paraphrase, entity, number or technical term, context-dependent question, and hard or mixed-intent query. Twenty-four queries form the development split and twelve form the holdout split. The holdout contains two queries from every category.

Labels use stable audio_slug, start_ms, and end_ms spans instead of database IDs. A result matches a gold span when it comes from the same episode and its temporal intersection covers at least 50% of the shorter interval. Recall@k measures the proportion of relevant spans retrieved; HitRate@k measures whether any relevant span was retrieved; MRR rewards the rank of the first relevant result. The current queries each have one primary gold span, so Recall@k and HitRate@k are numerically equal.

Holdout results

Strategy	Recall@1	Recall@3	Recall@5	MRR	Warm p95
Lexical	0.167	0.167	0.167	0.167	13 ms
Semantic	0.500	0.500	0.500	0.500	146 ms
RRF	0.583	0.583	0.583	0.583	77 ms
Hybrid	0.417	0.750	0.833	0.590	565 ms
Hybrid alternative	0.417	0.750	0.833	0.590	1,004 ms

The default hybrid retrieved 10 of 12 holdout answers within the top five. It improved Recall@5 by 0.333 absolute over semantic retrieval and by 0.250 over RRF. The pair-aware alternative changed raw scores for some queries but did not change any gold-answer rank in this evaluation, while its p95 latency was higher. The evidence therefore supports the simpler target-only hybrid as the shipped default.

The results should be interpreted at hackathon scale: twelve holdout queries are useful for catching large regressions but produce wide uncertainty, and every query changes the aggregate by 0.083. A production evaluation should contain substantially more queries, graded relevance judgments, and independent assessors.

Production considerations

Retrieval monitoring should include Recall@k, Precision@k, nDCG, MRR, zero-result rate, query mix, and relevance drift. Speech quality should be monitored separately with word error rate, diarization error rate or JER, speaker-count failures, and timestamp error. Operational measures include transcription failures and cost, ingestion throughput, p50/p95/p99 search latency, model CPU and memory, embedding throughput, and index size.

At larger scale, ingestion becomes an asynchronous job, embeddings and reranking are batched, metadata or tenant filters are applied before retrieval, and pgvector indexes are partitioned by tenant or corpus. HNSW recall should be sampled against exact scans. Cached transcripts and model artifacts allow subsequent demonstrations to run without hosted transcription.

Limitations

- Speaker labels identify voices but do not automatically resolve real names. The current files consistently use spk : 0 and spk : 1; a reviewed per-episode identity map would improve presentation.
- Transcription errors remain searchable errors. Proper-name examples include phonetic variants such as Geoffrey/Jeffrey, Jordan Noone/Noon, and Shaahin/Shahen. Entity aliases or corrected display metadata would improve exact name retrieval without rewriting the canonical transcript.
- The golden set is small and comes from one podcast format. Host introductions and summaries duplicate guest topics, creating realistic but difficult attribution cases.
- The temporal evaluator does not currently require a speaker match. Retrieval relevance and diarization accuracy should be reported as separate metrics.
- PostgreSQL FTS is not BM25, and the cross-encoder was trained on general passage ranking rather than podcast dialogue.
- The Q/A alternative depends on punctuation, speaker changes, and a seven-second gap heuristic. It does not yet group long multi-utterance responses.
- Latency was measured on one local reference machine with warm model caches. Cold model startup is substantially slower.
- Audio redistribution remains subject to the source material's licensing and the submitter's rights.

AI usage and coding-agent disclosure

ChatGPT and OpenAI Codex supported different stages of development. ChatGPT was used first as an architecture sounding board. The user prompted it to compare hosted and local transcription, utterance versus conversational-chunk retrieval, PostgreSQL FTS versus BM25 infrastructure, dense-only versus hybrid retrieval, weighted score fusion versus RRF and reranking, duplicate embedding storage, and neighbor-aware reranking. The user selected among those alternatives and converted the decisions into the implementation requirements supplied to Codex.

The central architecture decision from that brainstorming was to separate **retrieval granularity** from **evidence granularity**: overlapping speaker-aware chunks provide semantic context, while speaker-attributed utterances remain the returned evidence. Lexical and semantic systems nominate candidates without comparing incompatible raw scores; a local cross-encoder reranks their union, while RRF remains a baseline. The design deliberately avoided extra hackathon infrastructure such as OpenSearch solely for BM25, Ollama, LangChain/LangGraph, an LLM search agent, multiple databases, and duplicate utterance embeddings.

Codex then translated those requirements into the Docker Compose project, database schema and migrations, Gemini adapter, transcript normalization, chunking, local embedding and reranking code, API, UI, tests, evaluation tooling, and documentation. The user ran and reviewed the real transcription and ingestion workflow, reviewed the golden timestamp spans, identified the host-question ranking problem, and directed the separation of target-only hybrid from experimental hybrid_alt. This is also an example of an AI-proposed idea being revised after empirical review: symmetric neighbor context sounded useful during brainstorming but caused relevance leakage in real dialogue.

A fuller disclosure, including the architecture questions posed to ChatGPT, alternatives considered, decisions carried forward, Codex interaction trace, user responsibilities, and verification commands, is provided in docs/ai-usage.md.

Reproduction

```
cp .env.example .env          # add GEMINI_API_KEY only if retranscribing
make db-up
make migrate
make transcribe               # reuses matching canonical caches
make ingest
make up
make test
make evaluate
```

The Streamlit demo runs at `http://localhost:8501`; FastAPI documentation runs at `http://localhost:8000/docs`. The generated holdout reports are stored under `eval/reports/`.